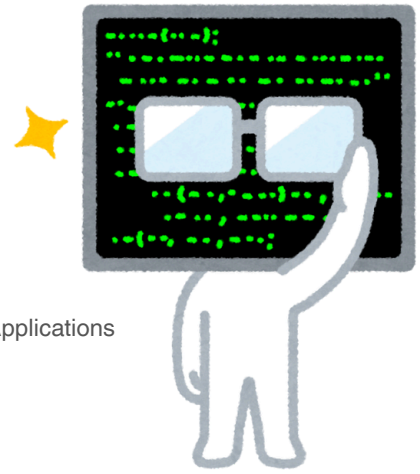
PROJECT REPORT

THE CONSTRUCTOR'S DILEMMA

AC++ObjectLifecycleGame

A Project on Object Oriented Programming in C++

Name	Hiten Kaushal
UID	25BCD10144
Section	25BCD-3A
Subject	OOPs in C++
Submitted to	Mr. Abhinay sood , Assistant Professor



Chandigarh University | Department of Computer Applications

Table of Contents

1. Abstract 3

2. Introduction and Problem Statement 3

3. Objectives 4

4. Features of the System 4

5. System Design (Classes and Responsibilities) 5

6. UML Class Diagram 5

7. OOP Concepts Used (with where they appear) 6

8. Core Algorithms 7

9. Sample Input / Output 9

10. Unique Selling Point (USP) 11

11. Limitations and Future Scope 12

12. Conclusion 12

References 12

1. Abstract

Learning C++isnot just about writing code - it is about understanding how code behaves. Among the most frequently misunderstood topics in Object-Oriented Programming is the object lifecycle: the exact sequence in which constructors and destructors execute when inheritance, composition and polymorphism are involved. Students regularly lose marks in examinations and interviews because they cannot predict this order correctly.

The Constructor's Dilemma is a text-based C++ game in which the player reads a short C++ program and predicts the exact output produced by constructor and destructor calls. After each answer, the game presents a step-by-step **Lifecycle Trace** that reveals the correct order with a one-line explanation for each step. The project is built entirely in standard C++ using inheritance, polymorphism, composition, encapsulation and file I/O, without external libraries. It covers five concept areas: constructor order, destructor order, composition, virtual destructors, and copy/assignment semantics.

Keywords: C++, OOP, constructors, destructors, object lifecycle, educational game, inheritance, polymorphism, file handling.

2. Introduction and Problem Statement

C++is a powerfulObject-OrientedProgramming language widely taught in undergraduate curricula. While students learn the syntax of classes, constructors and destructors, they often struggle with the semantics - specifically, the order in which these special member functions execute. Consider the following program:

```
class Vehicle { public: Vehicle(){cout<<"V";} ~Vehicle(){cout<<"~V";} };
};

class Engine { public: Engine(){cout<<"E";} ~Engine(){cout<<"~E";}
class Car : public Vehicle {
    Engine e;
public:
    Car(){cout<<"C";} ~Car(){cout<<"~C";}
};
int main() { Car c; }
```

The correct output is **V E C ~C ~E ~V**, but many students answer incorrectly, because the rule "base -> members -> body; destruction reverses" is easy to recite yet hard to apply. The Constructor's Dilemma is a console-based game that turns this concept into an interactive challenge. The project runs in a terminal, needs no graphics library, internet access or database, loads its questions from a plain text file and saves a high score to disk.

Problem statement. Students preparing for C++ examinations and interviews face three recurring problems:

1. **Passive learning.** Textbooks explain constructor order in prose, but students cannot test themselves interactively.
2. **No explanation of "why".** Online MCQ platforms mark an answer right or wrong but rarely explain the execution sequence step by step.
3. **Abstract concepts without visualisation.** Object lifecycle is invisible during normal coding, so misconceptions persist undetected.

The motivation was personal: during revision, the developer noticed that constructor/destructor order questions caused the most errors, yet no lightweight, self-contained tool existed to practise

them. Building a game that teaches while it tests addressed both the developer's learning need and a broader gap for fellow students.

Existing tools and their gaps

Tool	Strengths	Limitations
Textbooks (Let Us C++, The C++ Programming Language)	Authoritative, thorough	Passive, no interaction, no feedback on mistakes
Online MCQ platforms (e.g. GeeksforGeeks quizzes)	Large question banks	No step-by-step explanation; not locally runnable
Coding practice sites (HackerRank, LeetCode)	Real coding challenges	Rarely focus on lifecycle order; require internet
Console quiz projects (typical college projects)	Simple	Generic MCQs; no domain-specific teaching feature

No existing lightweight tool combines (a) lifecycle-specific questions, (b) interactive gameplay and (c) step-by-step execution traces in a single, offline, student-built application. This project fills that gap.

3. Objectives

1. To design and implement a fully functional, self-contained C++ game that tests understanding of object lifecycle.
2. To cover five concept areas: constructor order, destructor order, composition, virtual destructors, and copy/assignment semantics.
3. To provide an explanatory Lifecycle Trace after every question - the project's unique feature.
4. To demonstrate core OOP principles (inheritance, polymorphism, composition, encapsulation) and file I/O in a real application.
5. To keep the code portable - compiling and running on any standard C++ compiler without external dependencies.
6. To add motivational mechanics (lives, streak bonuses, high-score persistence) that sustain engagement.

4. Features of the System

- Shows a C++ code snippet with a question and four options (A, B, C, D).
- Verifies the answer and awards points for correct responses.
- Displays a step-by-step **Lifecycle Trace** after every answer, right or wrong.
- Starts with 3 lives; a wrong answer costs one life and resets the streak.
- Streak bonuses: +5 points at a 3-answer streak and +10 points at a 5-answer streak.
- Tracks score, lives, streak and per-concept statistics, and prints a final report with a verdict.
- Loads questions from lifecycle_questions.txt, editable by a non-programmer, with a built-in fallback bank if the file is missing.
- Saves and displays the highest score across sessions in highscore.txt.
- Ends when lives reach zero or all questions are answered; never crashes on a missing or malformed file.

Question categories

Category	Focus
CTOR	Constructor order in single classes, inheritance and composition
DTOR	Destructor order, especially reverse-declaration ordering of members
VIRTUAL	Why virtual ~Base() is essential for polymorphic deletion
COPY	Copy constructor vs. copy-assignment operator

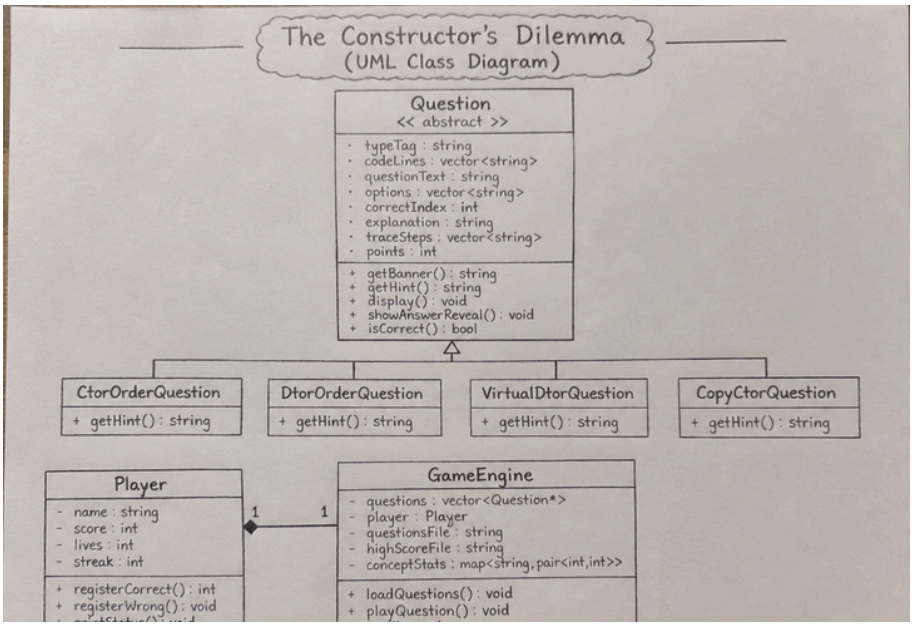
5. System Design (Classes and Responsibilities)

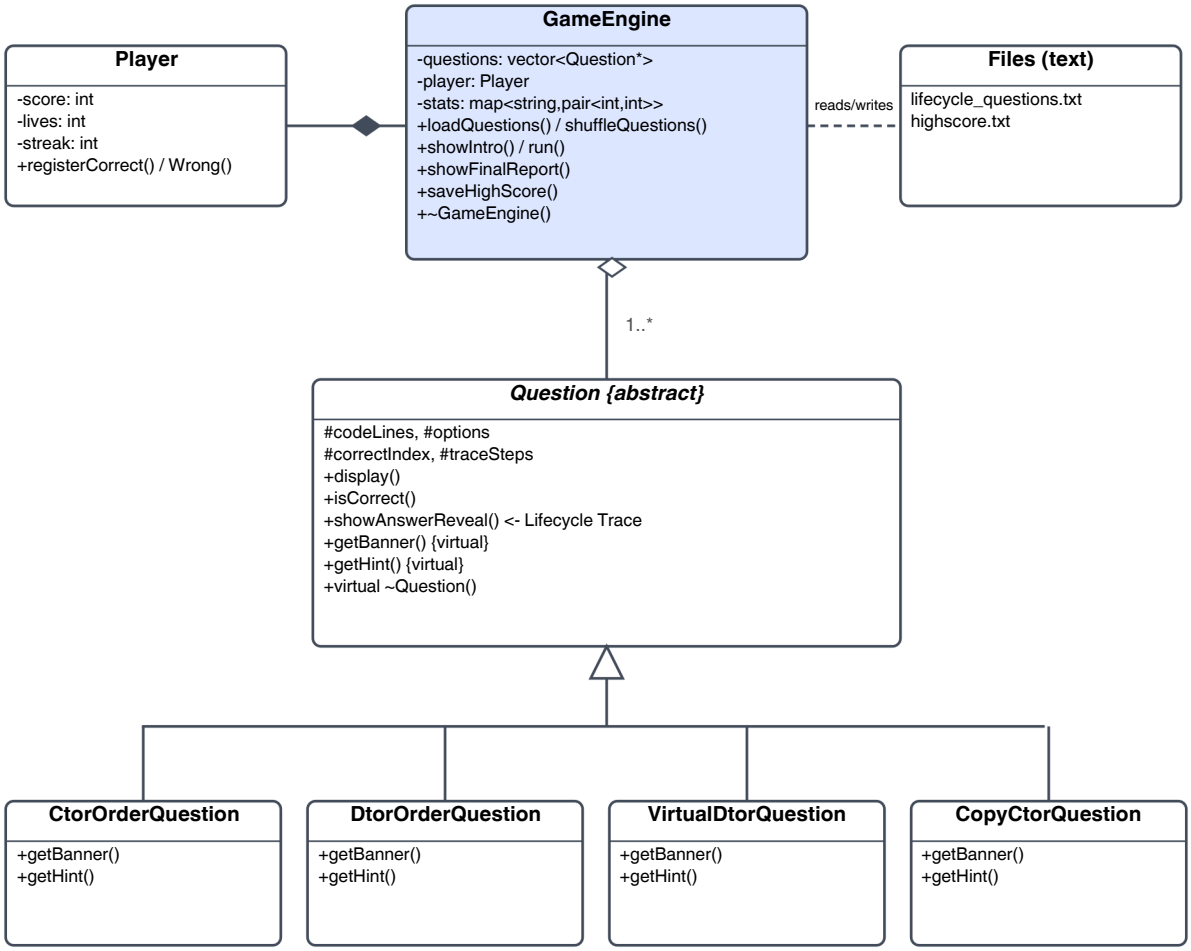
Class	Type	Responsibility
Question	Abstract base class	Common interface: display(), isCorrect(), showAnswerReveal(); holds code lines, options, correct index and trace steps; declares virtual getBanner() and getHint()
CtorOrderQuestion, DtorOrderQuestion, VirtualDtorQuestion, CopyCtorQuestion Player	Derived classes	Each overrides getBanner() and getHint() to add its own banner and hint for its concept area
	Concrete class	Stores score, lives and streak; registerCorrect() and registerWrong() apply the scoring rules
GameEngine	Manager class	Owns a vector of Question pointers and the Player; loads questions (file I/O), runs rounds, scoring, final report and high-score saving; its destructor deletes every Question
createQuestionByType()	Factory function	Maps a tag (CTOR, DTOR, VIRTUAL, COPY) to the correct derived object

The system follows a simple layered architecture: `main()` creates a `GameEngine`, which has-a Player and has-many Question objects.

6. UML Class Diagram

Thediagram belowshows theclasses, their key attributes and methods, and the relationships between them (inheritance, composition and association).





Filled diamond = composition | hollow triangle = inheritance | dashed line = association/file I/O

How to read the diagram

- **Inheritance:** CtorOrderQuestion, DtorOrderQuestion, VirtualDtorQuestion and CopyCtorQuestion inherit from the abstract class Question.
- **Composition:** GameEngine owns its Player and its collection (1..*) of Question objects; it deletes them in its destructor.
- **Association:** GameEngine reads lifecycle_questions.txt and reads/writes highscore.txt.
- In UML notation + means public, - means private and # means protected. Names in italics or marked {abstract} cannot be instantiated.

7. OOP Concepts Used (with where they appear)

The project is self-referential: it is a C++ program about C++ object lifecycles, and it models correct object lifecycle management in its own design. The table gives a quick map of the concepts used.

No.	OOP concept	Where it is used in the project	Benefit
1	Encapsulation	Player and Question keep data private/protected and expose only necessary methods	Data cannot be changed in an invalid way
2	Abstraction	Question provides a clean interface and hides internal representation	Hides details, exposes only what is needed

No.	OOP concept	Where it is used in the project	Benefit
3	Inheritance	Four question classes inherit from Question getBanner() and getHint() are virtual and overridden; called through Question*	Code reuse and a clear hierarchy
4	Polymorphism	GameEngine has-a Player and has-many Question*	One call, different behaviour per question type
5	Composition	The topic of the game; ~GameEngine() deletes all Question objects	Real "has-a" relationships
6	Constructors and destructors	ifstream / ofstream for question loading and high-score persistence	Safe cleanup, no memory leaks
7	File handling	vector<Question*>, map<string, pair<int,int>> for per-concept statistics	Editable data and saved progress
8	STL containers	#define SHOW_LIFECYCLE toggles the Lifecycle Trace	Less code, ready-made structures
9	Preprocessor directives		Feature toggling without logic changes

7.1 Abstract Base Class and Polymorphism

Questiondefinesthe common interface (display(), isCorrect(), showAnswerReveal()) and two virtual methods (getBanner(), getHint()) that every derived class overrides. A Question* pointer can therefore be used uniformly while each subclass adds its own banner and hint.

7.2 Factory Function

createQuestionByType() maps a string tag (CTOR, DTOR, VIRTUAL, COPY) to the correct derived object. This keeps parsing clean and makes the system extensible: adding a new question type only requires a new class and one line in the factory.

7.3 File Parser

Aline-based parser reads @-prefixed directives (@TYPE, @CODE, @QUESTION, @OPTION, @ANSWER, @EXPLAIN, @TRACE, @POINTS, @END) and populates a Question object. A fallback question bank is embedded in the source, so the game always runs even if the file is missing.

7.4 Feature Flag for the Unique Feature

A single#defineSHOW_LIFECYCLE1 at the top of the file controls whether the Lifecycle Trace is displayed. Setting it to 0 reverts the game to a plain quiz, demonstrating modular design and the use of preprocessor directives for feature toggling.

7.5 Score Persistence

saveHighScore()reads the previous best from highscore.txt, compares it with the current score and writes a new record if beaten, demonstrating both input and output file streams.

8. Core Algorithms

8.1 The Lifecycle Rule

Therule everyquestion tests is: **construction** runs Base -> Members (in declaration order) -> Body, and **destruction** is always the exact reverse. For two objects in main(), the later-created object is destroyed first.

8.2 Scoring and Lives

Event	Effect
Game starts	Player has 3 lives, score 0, streak 0
Correct answer	Base points (@POINTS) awarded; streak increases
3-answer streak	+5 bonus points
5-answer streak	+10 bonus points
Wrong answer	One life lost; streak resets
Lives reach 0 or questions exhausted	Game ends; final report and high-score check

8.3 Round Flow

```
Startup
 -> GameEngine constructed

 -> loadQuestions()       <- reads lifecycle_questions.txt
-> shuffleQuestions()
-> showIntro()

Per Round
 -> Question::display()
 -> read answer (A/B/C/D, or 0 to quit)
 -> Player::registerCorrect() / registerWrong()
 -> Question::showAnswerReveal()    <- Lifecycle Trace

End
 -> showFinalReport()
-> saveHighScore()        -> writes highscore.txt
```

8.4 Question Bank Format

Thefile `lifecycle_questions.txt` uses the following directives:

Directive	Meaning
@TYPE	Question type: CTOR, DTOR, VIRTUAL or COPY
@CODE	One line of the code snippet (repeatable)
@QUESTION	The question text
@OPTION	One answer option (repeatable; up to four) Correct
@ANSWER	option number (1-4)
@EXPLAIN	One-line explanation shown after answering
@TRACE	One step of the Lifecycle Trace (repeatable) Points
@POINTS	awarded for a correct answer
@END	Marks the end of the question

Example block:

```
@TYPE CTOR
@CODE class Demo { public: Demo(){cout<<"C";} ~Demo(){cout<<"D";} };
@CODE int main() { Demo a; Demo b; }
@QUESTION What is the exact output?
@OPTION C D C D
@OPTION C C D D
@OPTION D D C C
@OPTION C D D C
@ANSWER 2
@EXPLAIN Objects are destroyed in the reverse order of creation.
@TRACE [1] Demo a; -> ctor -> C
@TRACE [2] Demo b; -> ctor -> C
@TRACE [3] End of main: b destroyed -> D
@TRACE [4] Then a destroyed -> D
@POINTS 10
@END
```

9. Sample Input / Output

9.1 Requirements and How to Run

Item	Requirement
Language	C++ (C++11 or later)
Compiler	g++ (GCC / MinGW), or MSVC with C++11 support
Operating system	Windows, Linux or macOS
Editor / IDE	Code::Blocks, Dev-C++, VS Code, or any text editor
Hardware	Any system capable of running a console application
Source file	ConstructorDilemma.cpp (about 500 lines, standard library only)

To compile and run from a terminal:

```
g++ -std=c++11 ConstructorDilemma.cpp -o ConstructorDilemma
./ConstructorDilemma            (on Windows: ConstructorDilemma.exe)
```

9.2 Sample Session

```
  ############################################################
  #   T H E   C O N S T R U C T O R ' S   D I L E M M A   #
  #    AC++ObjectLifecycleGame    #    #    Projectby:HitenKaushal
  (25BCD10144)  # # Class :25BCD-3A  # # Guidedby :AbhinaySir  #
  ############################################################



  [+] Loaded 10 questions from 'lifecycle_questions.txt'.

  Enter your name (or press ENTER for 'Hiten'): Hiten
```

```
============================================================
 Q1   | CONSTRUCTOR ORDER | 10 pts
============================================================
  Code:
    classBase     {public:Base(){cout<<"B";} ~Base(){cout<<"~B";} };
    classDerived:publicBase{public: Derived(){cout<<"D";}
                                    ~Derived(){cout<<"~D";} };
    int main() { Derived d; }

  What is the exact output?
    A) B D ~B ~D
    B) D B ~D ~B
    C) B D ~D ~B
    D) D B ~B ~D

  [Hint] Base first -> then members (declaration order) -> then body.

  Player: Hiten | Score: 0 | Lives:3       |  Streak: 0
  Your answer (A/B/C/D, or 0 to quit):C

  [OK] Correct! +10 pts
  Correct answer: C) B D ~D ~B

  --Lifecycle Trace ------------------------------------------
  [1] Derived d; -> Base ctor runs first -> B
  [2] Then Derived body -> D
  [3] End of main: Derived dtor -> ~D
  [4] Then Base dtor -> ~B
  ------------------------------------------------------------
  Why: Base constructor always runs before the derived constructor.
       Destruction is the exact reverse.

  Press ENTER to continue...
```

9.3 Final Report

```
============================================================
                     GAME OVER
============================================================
Player: 95        Questions Answered: 10

 Correct:             8
 Accuracy:            80.0%
 Lives Remaining:     1

  --Concept Breakdown ----------------------------------------
    COPY        2 / 2
    CTOR        3 / 4
    DTOR        1 / 2
    VIRTUAL     2 / 2

 Verdict: C++ Object Lifecycle Master!
============================================================
* NEW HIGH SCORE saved: 95 pts (was 0)
```

9.4 Testing

Test type Unit-level	Description
	Each question type instantiated and verified against known correct output
File handling	Tested with (a) valid file, (b) missing file, (c) empty file, (d) malformed lines
Edge cases	Invalid input (e.g. X, blank line), early quit (0), zero lives, all questions answered
Persistence	High-score file checked after consecutive runs
Toggle	Built and tested with SHOW_LIFECYCLE set to both 1 and 0

10. Unique Selling Point (USP)

The LifecycleTrace: aQuizThatTeaches While It Tests

After every question, the game does not merely say "Correct" or "Wrong". It walks the player through the exact execution sequence of the code shown, with each step numbered:

```
-- Lifecycle Trace ----------------------------------------

[1] Base Vehicle constructor          ->  V
[2] Member Engine constructor         ->  E
[3] Car body                          ->  C
[4] Car destructor                    -> ~C
[5] Member Engine destructor          -> ~E
[6] Base Vehicle destructor           -> ~V
-----------------------------------------------------------
Why: Construction: Base -> Members (declaration order) -> Body.
Destruction is always the exact reverse.
```

This is what distinguishes the project from a conventional quiz:

1. **Every answer is a learning moment.** Right or wrong, the player sees why the order occurs.
2. **It makes an invisible behaviour visible.** The trace converts hidden compiler behaviour into a visible, ordered sequence.
3. **It is configurable.** The SHOW_LIFECYCLE flag turns the trace on or off without changing any other logic.
4. **It is easy to extend.** Trace steps are written in the question file with @TRACE, so anyone can add new questions with their own traces.
5. **Motivation built in.** Lives, streak bonuses and a persistent high score encourage replay.

Feature	Ordinary quiz / MCQ site	The Constructor's Dilemma
Feedback	Right or wrong only	Step-by-step Lifecycle Trace with explanation
Focus	Generic questions	Object lifecycle: constructor/destructor order
Availability	Often needs internet	Offline, single standard-C++ program
Questions	Fixed by the platform	Editable text file by a non-programmer
Progress	Lost when closed	High score saved to disk
Adding a question type	Edit many places	One new class and one factory line

11. Limitations and Future Scope

Current limitations

- Console-only interface; no graphical visualisation.
- No multiplayer or online leaderboard.
- The parser expects a strict @-directive format; malformed files fall back to the built-in bank.
- No adaptive difficulty - questions are shuffled, not selected based on performance (though per-concept stats are tracked).

Future scope

1. Graphical interface using Qt or SFML for an animated constructor/destructor "timeline".
2. Adaptive difficulty that re-asks questions from weak concepts, using the existing conceptStats map.
3. Timer mode that adds a per-question countdown for extra challenge.
4. Expanded question bank covering templates, move semantics, static members and exception safety.
5. Multiplayer / leaderboard using file-based or network-based score storage.
6. Web version using WebAssembly, so the same C++ logic runs in a browser.

12. Conclusion

The Constructor's Dilemma demonstrates that a college project can be simultaneously simple to build, rich in OOP concepts and genuinely useful. By targeting a specific, well-known weakness in how students learn C++ - object lifecycle order - the project turns a passive revision topic into an interactive challenge.

The use of inheritance, polymorphism, composition, encapsulation and file handling mirrors what is taught in the classroom, while the Lifecycle Trace adds an original educational dimension that no standard quiz provides. The project is lightweight, portable and extensible, and can be used directly by students preparing for examinations and interviews. Above all, it proves a simple idea: the best way to understand code is to see exactly what it does, step by step.

References

1. BjarneStroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.
2. Yashavant Kanetkar, Let Us C++, 10th Edition, BPB Publications, 2019.
3. Herbert Schildt, C++: The Complete Reference, 4th Edition, McGraw-Hill, 2002.
4. cppreference.com - "Constructors and member initializer lists" and "Destructors".
5. ISO/IEC 14882:2011 - Programming Languages - C++.
6. GeeksforGeeks - "Order of Constructor/Destructor Call in C++".